

VERİ KODLAMA

[Kaynak](#)

İÇERİK

1. Veri Gösterimi ve Karakter Kodlaması
2. ASCII (American Standard Code for Information Interchange - Amerikan Bilgi Değişimi Standart Kodu)
3. Karakter Kodlaması: ASCII'den Unicode'a Evrim

Veri Gösterimi ve Karakter Kodlaması

Bilgisayar sistemlerinin temelinde her türlü veri, ikili sayı sistemindeki basamaklar (**bits**) olarak temsil edilir ve bu bit yığınları analiz kolaylığı açısından sıklıkla on altılı (**hexadecimal**) sayı sisteminde gruplandırılır. Renkler, komutlar veya bellek adresleri gibi metin tabanlı veriler de sistemin en alt katmanında tamamen sayılardan ibarettir. Peki, donanım düzeyinde her şey sayılardan oluşuyorsa, bu sayılar nasıl harflere, noktalama işaretlerine veya emoji'lere dönüşmektedir?

Bu sorunun cevabı, sistemler arası iletişimin temelini oluşturan **Encoding** (Kodlama: İnsan tarafından okunabilir karakterlerin ve sembollerin, bilgisayar tarafından işlenebilmesi ve depolanabilmesi için önceden üzerinde anlaşılmış belirli sayısal değerlere dönüştürülmesini sağlayan standartlaştırılmış haritalama protokolü) kavramında yatmaktadır. Bilgisayar terminolojisinde bir karakter, aslında sistemlerin anlamı üzerinde mutabık kaldığı salt bir sayıdır.

ÖNEMLİ: Teknik düzeyde **Representation** ve **Encoding** kavramları sıklıkla birbirine karıştırılır, ancak aralarında net bir mantıksal ayrım vardır:

- **Representation** (Temsil), verinin bellekte (memory) bitler ve sayılar olarak fiziksel/mantıksal olarak nasıl tutulduğu gerçeğidir. Verinin varoluş biçimidir.
- **Encoding** ise bu sayıların spesifik olarak hangi anlama geldiğini belirleyen haritalamadır (Örneğin, bellekteki belirli bir sayısal değerın ekrana "A" harfi olarak yansıtılması gerektiği kuralı).

İnternette indirilen bir altyazı dosyasını, bir belgeyi veya bir web sayfasını açtığınızda anlamsız karakter yığınlarıyla (gibberish) karşılaşmanızın arkasındaki teknik neden tam olarak budur. Veri bütünlüğünde bir bozulma yoktur; sorun, dosyayı oluşturan ve kaydeden kullanıcının sistemi ile, o dosyayı okumaya çalışan kullanıcının sisteminin bellekteki aynı sayılara farklı **Encoding** standartlarıyla bakmasıdır. İlk sistem bellekteki sayı dizisini bir karaktere çevirirken kullandığı tabloyu (örneğin UTF-8), ikinci sistem farklı bir tabloyla

(örneğin ASCII veya ISO-8859-1) yorumlamaya çalışıldığında veri ekrana anlamsız semboller dizisi olarak yansır.

Yazılımsal açıdan bakıldığında metin içerisindeki her bir karakter (örneğin `q`, `5` veya `!`), spesifik bir sayısal koda atanmıştır. Dolayısıyla bir **string** (Karakter dizisi: Birden fazla karakterin bir araya gelerek oluşturduğu, genellikle metin verilerini tutmak için kullanılan veri tipi), aslında bilgisayarın bellekte sakladığı ve tıpkı diğer sayısal veriler gibi işleyip yönlendirdiği ardışık bir sayılar dizisinden ibarettir. Siber güvenlik analizlerinde, özellikle zafiyet sömürüsü veya zararlı yazılım analizinde (malware analysis) bellekteki bir **string** verisinin aslında bir sayı dizisi olduğunu kavramak, veriyi manipüle edebilmek için hayati bir öneme sahiptir.

ASCII (American Standard Code for Information Interchange - Amerikan Bilgi Değişimi Standart Kodu)

Bilgisayar sistemlerinin donanım seviyesinde yalnızca 0'lar ve 1'lerden (ikili sistem - **Binary**) anladığını daha önce netleştirmiştik. Peki donanım yalnızca `0` ve `1` sinyallerini işleyebiliyorken, örneğin "TryHackMe" gibi bir metni bir dosyaya nasıl kaydeder ve ekranda nasıl görüntüleriz? Diskte bu dosyanın fiziksel karşılığı nedir?

Bu sorunun çözümü, standartlaştırmada yatar. Sistemin `T`, `r`, `y` gibi harflerin her birini temsil eden bit dizilimleri üzerinde mutabık kalması gerekir. Örneğin, dünya üzerindeki tüm sistemler `T` karakterini `01010100` bit dizilimiyle temsil etmek üzere programlanırsa, bilgisayar bellekte veya diskte bu dizilimi okuduğunda bunun bir `T` olduğunu tereddütsüz anlar. Elbette bu mutabakatın sadece bir harf için değil; alfabedeki tüm harfler, rakamlar, noktalama işaretleri ve sistem kontrol karakterleri (Control Characters) için de yapılması şarttır. İşte bilgisayar üreticilerinin, donanım tasarımcılarının ve yazılımcıların üzerinde anlaştığı bu ilk ve en temel standartlardan biri **ASCII**'dir.

ASCII Mimarisi ve Teknik Sınırları

1963 yılında geliştirilen **ASCII** (Bilgi Değişimi İçin Amerikan Standart Kodlama Sistemi), İngilizce harfleri, rakamları, noktalama işaretlerini ve yazıcı/terminal kontrol karakterlerini temsil etmek için **0 ile 127** arasındaki sayıları kullanan erken dönem bir karakter kodlama (Encoding) standardıdır.

ÖNEMLİ: ASCII başlığındaki "A" (American) harfi teknik bir kısıtlamanın da işaretidir. Bu standart doğrudan İngilizce alfabeyi temel alır. Orijinal ASCII mimarisi tam olarak **7-bit** uzunluğundadır ($2^7 = 128$ benzersiz karakter). Veriler bellekte 8-bit'lik (1 Byte) bloklar halinde tutulduğundan, orijinal ASCII kullanıldığında byte'ın en anlamlı biti (Most Significant Bit - MSB) genellikle 0 olarak bırakılır veya ağ iletiminde hata denetimi (Parity Bit) amacıyla kullanılırdı.

Aşağıda orijinal 128 karakterlik ASCII tablosunun teknik işleyişi kavramak için seçilmiş spesifik bir kesiti bulunmaktadır:

Decimal (Onluk)	Hexadecimal (On Altılı)	Binary (İkili)	Sembol	Açıklama
48	30	00110000	0	Sıfır (Rakam)
57	39	00111001	9	Dokuz (Rakam)
65	41	01000001	A	Büyük A
88	58	01011000	X	Büyük X
90	5A	01011010	Z	Büyük Z
91	5B	01011011	[	Açık Köşeli Parantez
97	61	01100001	a	Küçük a
122	7A	01111010	z	Küçük z
127	7F	01111111	DEL	Delete (Kontrol Karakteri)

Siber Güvenlikte ASCII'nin Mantıksal Analizi:

Tabloyu incelediğimizde iki kritik mühendislik detayı göze çarpar:

- Sıralı (Sequential) Yapı:** Harfler ve rakamlar rastgele değil, sıralı bir mantıkla dizilmiştir. a , b , ve c sırasıyla **61, 62, 63 (Hex)** değerlerini alır. Siber güvenlikte (özellikle Reverse Engineering veya Fuzzing süreçlerinde) bellekte 41 (Hex) değerini gördüğümüzde bunun A harfi olduğunu ve bu dizilimin peş peşe gitmesi durumunda (41 41 41) bir **Buffer Overflow** (Bellek Taşması) zafiyeti tespiti için gönderilen standart test payload'u olduğunu anlarız.
- Karakter-Benzersizlik İlkesi:** Her bir sembolün (örneğin [) mutlak ve benzersiz bir Hexadecimal (5B) karşılığı vardır. İşlemci metin okumaz; diskten 5B değerini okur ve video bağdaştırıcısına (GPU/Monitör) [çizmesini emreder.

Hafıza İncelemesi: "TryHackMe" Örneği

Bir metin editörü açıp "TryHackMe" yazıp dosyayı file.txt olarak kaydettiğimizde diskte (bit seviyesinde) tam olarak ne olur? Dosyanın saf **Binary** (İkili) görünümü şu şekildedir:

```
01010100 01110010 01111001 01001000 01100001 01100011 01101011 01001101
01100101 00001010
```

Yukarıdaki çıktı, dosyanın hard diskteki manyetik veya flash hücrelerindeki fiziksel karşılığıdır. İnsan okumasına uygun değildir. Editör bu dosyayı açtığında, bu bit yığını ASCII standardına göre 8'erli bloklar halinde (son bitleri değerlendirerek) ayrıştırır ve ekrana şu karakterleri basar: T r y H a c k M e \n

DİKKAT: En sondaki 00001010 dizilimi (Hex karşılığı 0a), metin sonundaki Enter tuş vuruşunu temsil eden \n (**Line Feed - Yeni Satır**) kontrol karakteridir. Ağ protokollerinde

(Örn: HTTP) ve log analizlerinde bu karakterler (`0a` ve `0d` - Carriage Return) paketlerin veya satırların nerede bittiğini belirlediği için, bu değerlerin manipüle edilmesi **CRLF Injection** gibi ciddi zafiyetlere yol açar.

Bitleri doğrudan okumak hata yapma riskini artırdığı için, analizlerimizde her 4 biti 1 Hexadecimal karaktere gruplayarak dosyayı okuruz (Hex Dump):

```
54 72 79 48 61 63 6b 4d 65 0a
```

Bu çıktı, aynı dosyanın Hexadecimal temsilidir. (Örn: `54` = `T`, `72` = `r`, `0a` = `\n`).

Decimal (Onluk) sistem (`124 162... vb.`) teknik analizlerde bellek adreslerini veya bitleri temsil etmek için elverişsiz olduğundan sektör standartlarında karakter analizleri daima **Hexadecimal** üzerinden yapılır.

7-Bit Sınırı ve Genişletilmiş Encoding Standardı İhtiyacı

ASCII, Amerikan İngilizcesi için kusursuz çalışsa da küresel iletişim için yetersiz kalmıştır. İspanyolca (ñ, ¿), Almanca (ß, ü), Lehçe (ł, ó) veya Türkçe (ş, ğ, ç) gibi dillerdeki özel karakterler 128 karakterlik orijinal tabloya sığmaz.

Orijinal ASCII'nin 7 bit kullandığını belirtmiştik. Boşta kalan 8. biti de kullanıma dahil ettiğimizde ($2^8 = 256$), ekstra **128 karakterlik** yeni bir alan kazanırız (Genişletilmiş ASCII). Ancak bu ekstra 128 alan bile dünyadaki tüm Avrupa dillerini aynı anda kapsamak için yeterli değildir.

Bu donanımsal ve mantıksal sınırı aşmak için **ISO/IEC 8859** serisi uluslararası standartlar geliştirilmiştir. Bu mimaride, ilk 128 karakter (0-127) daima standart ASCII ile aynı kalırken, son 128 karakter (128-255) seçilen dile göre farklı sembollere haritalanmıştır:

- **ISO-8859-1 (Latin-1):** Batı Avrupa dilleri (Almanca ß/ü, Fransızca é/ç, İspanyolca ñ, İskandinav dilleri).
- **ISO-8859-2 (Latin-2):** Orta ve Doğu Avrupa dilleri (Lehçe ł/ó, Çekçe č/ř, Macarca ő, Romence ș).

Zafiyet ve Hata Vektörü (Encoding Mismatch):

Siber güvenlikte ve sistem yönetiminde en sık karşılaşılan "metin bozulması" (gibberish/mojibake) sorunlarının kök nedeni buradadır. Eğer bir kullanıcı belgeyi `ISO-8859-1` formatında kaydederse (örneğin belleğe İspanyolca ñ harfi için `F1` Hex değerini yazar), belgeyi okuyan diğer kullanıcının sistemi bu dosyayı `ISO-8859-2` standardı ile açmaya çalışırsa, sistem `F1` değerini alır ancak kendi tablosunda bu değer Lehçe ł harfine denk geldiği için ekrana yanlış karakter basar. Bellekteki veri değişmemiştir; değişen, o verinin hangi **Encoding** sözlüğü ile okunduğudur.

Karakter Kodlaması: ASCII'den Unicode'a Evrim

Önceki aşamalarda ASCII'nin, İngilizce harfleri, rakamları ve temel noktalama işaretlerini kapsayan 128 karakterlik, **7-bit** mimarisine sahip bir standart olduğunu incelemiştik. Ancak 7-bitlik bu alan; İspanyolca ñ, Euro sembolü €, Japonca あ veya Arapça ب gibi karakterleri barındırmak için teknik olarak yetersizdi. Bu kapasite sorununu çözmek adına **Extended ASCII** (Genişletilmiş ASCII) adı altında 8. bit kullanıma sokularak toplam kapasite 256 karaktere (2^8) çıkarılmış ve bölgesel varyantlar (**ISO-8859-1**, **ISO-8859-2**, **Windows-1252** vb.) oluşturulmuştur.

Siber güvenlik perspektifinden bakıldığında bu durum, sistemler arası veri bütünlüğü açısından tam bir kaosa neden olmuştur. Eğer bir veri paketi veya belge **ISO-8859-1 (Latin 1)** standardı ile kodlanıp belleğe 0 karakteri olarak yazılırsa ve alıcı sistem bu veri bloğunu **ISO-8859-2 (Latin 2)** standardıyla çözmeye (decode) çalışırsa, ekranda 0 karakteri belircektir. Bu senaryo bize, gönderici ve alıcının mutlak suretle aynı **Encoding** (Kodlama) algoritmasını kullanmak zorunda olduğunu, aksi takdirde verinin yanlış yorumlanacağını (veri manipülasyonu olmasa bile bütünlüğün bozulmuş gibi görüneceğini) kanıtlamaktadır.

Kapasite Krizinin Teknik Boyutu

İngilizce, büyük ve küçük harfler dahil olmak üzere toplam 52 karaktere ihtiyaç duyarken, küresel dillerde bu sayı logaritmik olarak artar:

- **Arapça:** Farklı bitişik yazım formları (ligatures) ve harekelendirmeler (diacritics) nedeniyle 250'den fazla karaktere ihtiyaç duyar.
- **Japonca:** Eğitim Bakanlığı standartlarına göre günlük kullanımda 2.136 Kanji (logografik karakter) bulunur. Teknik standart olan **JIS X 0208**, 6.879 karakter tanımlar.
- **Çince:** Modern **GB 18030-2022** standardı, 87.887'den fazla Hanzi (Çince karakter) tanımlamaktadır.
- Tüm bu metin karakterlerinin üzerine, günümüz dijital iletişiminin ayrılmaz bir parçası olan emojiler de eklendiğinde, 8-bitlik bölgesel çözümlerin tamamen yetersiz kaldığı matematiksel bir gerçektir.

Unicode: Evrensel Çözüm

Bu donanımsal ve mantıksal çıkmaz, **Unicode** standardının doğmasına zemin hazırlamıştır. Unicode, dünya üzerindeki tüm modern ve tarihi yazım sistemlerindeki her bir karaktere benzersiz bir sayısal değer (**Code Point**) atayan evrensel bir karakter kodlama standardıdır. Unicode sayesinde sistemler, "bu dosya hangi dilde ve hangi kodlama ile yazılmış?" sorusunu sormaktan kurtulur; tek bir dosya veya payload içerisinde birden fazla dil ve sembol aynı anda, bozulma riski olmadan kullanılabilir.

Her karakter mutlak bir sayısal koda (Code Point) sahiptir. Örneğin:

```
U+0041 = Latin "A"  
U+03A9 = Yunan "Ω"  
U+3042 = Japon Hiragana "あ"
```

Yukarıdaki `U+` ön eki, değerin bir Unicode Code Point olduğunu; ardındaki değer ise bu karakterin Hexadecimal (On altılı) karşılığını temsil eder. Güncel **Unicode 17.0** standardı, yaklaşık 4.000'i emoji olmak üzere 157.000'e yakın karakteri tanımlamaktadır.

Unicode İmplementasyonları: UTF-8, UTF-16 ve UTF-32

Unicode bir haritalama standardıdır (hangi sayının hangi karakter olacağı). Ancak bu sayıların bellekte (RAM) veya diskte fiziksel olarak **kaç byte** alan kaplayacağı, seçilen **UTF** (Unicode Transformation Format) implementasyonuna göre değişir. Zafiyet araştırmalarında veya ağ trafiği analizlerinde bu byte dizilimlerinin nasıl yapılandığını bilmek kritik önem taşır.

UTF-8 (Dinamik ve Yaygın Standart)

Modern web altyapısının ve ağ protokollerinin varsayılan standardıdır. Karakterin karmaşıklığına göre bellekte **1 ila 4 byte** arasında dinamik olarak yer kaplar.

ÖNEMLİ: UTF-8'in sektör standardı olmasının temel teknik nedeni **Geriye Dönük Uyumluluktur (Backward Compatibility)**. Orijinal ASCII tablosundaki ilk 128 karakter (`U+0000` ile `U+007F` arası), UTF-8'de de tam olarak **1 byte** yer kaplar ve bit dizilimleri ASCII ile birebir aynıdır. Böylece sadece İngilizce karakterler içeren eski bir sistem, UTF-8 ile sorunsuz çalışır.

Ancak `Ω` (`U+03A9`) gibi standart dışı karakterler **2 byte**, `🔥` (`U+1F525`) gibi emoji'ler veya karmaşık logogramlar ise **4 byte** alan kullanır. Bu dinamik yapı, belleğin maksimum verimlilikle kullanılmasını sağlar.

UTF-16

Karakterleri sabit olarak **2 veya 4 byte** bloklar halinde kodlar. Temel Latin, Kiril veya Çin Hanzi karakterleri 2 byte içine sığarken; nadir semboller veya emoji'ler iki adet 16-bitlik bloğun birleşimiyle (Surrogate Pairs) toplam **4 byte** yer kaplar. Örneğin `A` harfi `U+0041` olarak tutulurken, `🔥` emoji'si `U+D83D U+DD25` şeklinde iki parça halinde belleğe yazılır.

UTF-32 (Sabit ve İsrafkâr)

En basit ayrıştırma mantığına sahip olan ancak bellek yönetimi açısından en verimsiz yöntemdir. Karakterin ne olduğuna bakılmaksızın (ASCII `A` harfi dahil) istisnasız her karakter için bellekte **4 byte** (32-bit) alan tahsis eder.

Aşağıdaki blok, farklı karakterlerin bellek ve donanım seviyesindeki yansımalarını göstermektedir:

```
Karakter: 龍 (Ejderha – Kali Linux gibi ofansif dağıtımlarda sıkça görülür)
UTF-16 Code Point: U+9F8D
UTF-32 Code Point: U+00009F8D (Başında 4 byte'ı doldurmak için sıfırlar bulunur)
```

Karakter: 😊 (Gülen Yüz)

UTF-32 Code Point: U+0001F60A

Binary (İkili) Temsil: 0000 0000 0000 0001 1111 0110 0000 1010

Yorum: İşlemci bellekte bu 32-bitlik dizilimi okuduğunda, GPU'ya bir gülen yüz render etmesi komutunu gönderir.

Karakter: ♠ (Siyah Satranç Atı)

Unicode Code Point: U+265E

Binary (İkili) Temsil: 0010 0110 0101 1110

*Siber güvenlik analizlerinde, zararlı yazılımlar genellikle güvenlik duvarlarını veya IDS/IPS (Saldırı Tespit/Engelleme Sistemleri) mekanizmalarını atlatmak için bu UTF farklılıklarını kullanır. Zararlı bir **Payload**, sistem tarafından UTF-8 olarak analiz edilip zararsız bulunurken, hedef uygulamanın belleğinde UTF-16 olarak işlenerek (Encoding Confusion/Smuggling) zafiyeti tetikleyebilir. Bu nedenle bellek seviyesinde baytların hangi formata dönüştürüldüğünü anlamak hayati önem taşır.*